

Execution Under Rules: The Architectural Identity of the Cell Layer in the Coordination Knowledge Substrate Pattern

A derivation note from the Coordination Knowledge Substrate (CKS) pattern.

Author: Wenxin Li (Independent Researcher)

ORCID: 0009-0004-8065-3235

Date: 2 May 2026

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to articulate, in operational form, the architectural identity of the cell layer as the source paper commits to it — what a cell is, what it commits to, and what it does not do — independent of the substrate layer's role and independent of the boundary-crossing operations between them, so that downstream work can derive from, extend, or argue against the cell layer's identity without ambiguity.

Abstract

The CKS pattern is built from two structural layers — substrate and cell — and the parent derivation note treats the boundary between them as a joint architectural commitment. The cell side of that boundary, however, has independent architectural content that holds regardless of which substrate the cell operates on. This note formalizes the cell layer's architectural identity as standalone: four commitments (rule-governed behavior, bounded scope, substrate mediation, statelessness across executions) jointly sufficient and individually necessary for an execution context to qualify as a CKS cell; the activities the cell does not perform; the four adjacent design objects the cell is commonly conflated with but is not; the failure modes that violate cell identity; and an operational test for whether a given execution context is a CKS cell. Substrate-side commitments are treated in the companion standalone note; boundary-crossing operations are treated in the next note in the decomposition.

1. Why the cell layer's standalone identity needs to be formalized

The parent derivation note treats the substrate-cell boundary as an integrated commitment, defining each layer relative to the other. That treatment is right for the boundary as a whole, but several deployment cases require the cell layer's content to stand on its own.

The first is reuse and independent evaluation. Cell designs are commonly applied to multiple substrate instances — the same pattern operating on different substrate content — and cells are commonly evaluated for compliance independently of the substrates they operate on. Both require that cell identity be specifiable without reference to any particular substrate; if cell identity were relative to a specific substrate, the same pattern operating on different content would be a different cell, and a conformance audit would have to retrace the entire boundary relationship for every cell instance.

The second is the failure mode the cell layer most often suffers in practice: misreading as autonomous-agent territory. Implementations that treat the cell as "an autonomous AI agent that operates on data" lose the architectural content of the four commitments — particularly rule-governed behavior and statelessness across executions — and produce systems that look CKS-shaped but fail cell-side commitments under specific conditions. Naming the cell's standalone commitments is what makes such failures identifiable as failures of cell identity rather than as design choices.

A third is the prior-art posture this series serves: patentable derivations that focus on cell features are more defensibly contested when the cell layer's architectural content is publicly formalized as standalone.

2. What the cell layer is

In the CKS pattern, a **cell** is the bounded execution context in which work happens — where requests come in, substrate content is read, rules apply, decisions are made, and outputs are produced as substrate writes. The source paper defines a cell as "a coordinated collection of substrates, possibly just one for a simple task, together with human-authored orchestration rules that determine how the content is used when the cell executes" (§2.1). This standalone treatment formalizes the cell as an architectural object with four load-bearing commitments.

(a) Rule-governed behavior. A cell's behavior is determined by orchestration rules authored by humans (§2.3), not by ad-hoc LLM judgment, hardcoded logic outside the rule layer, or emergent agent behavior. Rules specify what the cell does in response to inputs, how it reads from the substrate, how it resolves contradictions at execution time, and what it writes back. The architectural commitment is that all cell behavior is traceable to a rule humans authored and govern; behavior with no rule trace is behavior outside the architecture.

(b) Bounded scope. A cell operates over a defined slice of substrate content for a defined task. Boundedness is not a performance optimization; it is an architectural commitment that distinguishes cells from systems maintaining global awareness or operating over arbitrary substrate content. A cell's scope is part of its orchestration rules; an execution context that operates over arbitrary substrate content is a different architectural object.

(c) Substrate mediation. A cell reads from the substrate as its primary source of state and writes its outputs back to the substrate; all coordination state the cell touches passes through the substrate. This commitment is what connects the cell to the AI-as-substrate-mediator role formalized as a separate derivation in this series. When a cell uses an LLM, the LLM operates within the cell as substrate mediator. When a cell uses no LLM — a deterministic cell that operates over substrate content under

rules without any LLM is still a cell — the same mediation pattern applies at the cell level. Substrate mediation is the cell's commitment, not the LLM's; the LLM, when present, inherits it as part of its mediator role.

(d) Statelessness across executions. A cell does not hold substrate-relevant state between invocations. State that should persist across cell executions lives in the substrate; state that exists only during a single execution is cell-internal and is discarded when execution completes. This commitment produces the linear-cost scaling property at the cell level (§6.1, §6.3) — cell cost is per-execution and scope-proportional, not size-proportional, because cells do not accumulate per-substrate-element state — and the substrate-as-source-of-truth property at the cell-coupling level (§11.3): substrate state is authoritative because cells do not hold competing state externally.

The four commitments are jointly sufficient and individually necessary for cell-layer identity. An execution context that satisfies all four is a CKS cell; one that satisfies fewer is something else — possibly useful, but not a cell in the architectural sense.

3. What the cell layer does not do

The four commitments imply six restrictions, several of which name failure modes more vividly than the positive form does.

The cell does not store coordination state independently of the substrate; state persistence is a substrate-layer responsibility, and a cell that maintains a "memory" outside the substrate — caching content, accumulating context, remembering decisions — is doing substrate-layer work in cell clothing. The cell does not exercise governance authority over substrate content; cell writes are rule-mediated, not authority-mediated, and inspect/modify/override rights remain with humans. The cell does not author or modify orchestration rules; rule authoring is a human exercise at design time, and a cell that modifies its own governing rules has crossed into rule authorship without human authority.

The cell does not communicate directly with other cells; cell-to-cell paths run through the substrate where they remain addressable and governable, and direct channels would bypass the substrate's source-of-truth and path-retraceability commitments (§3.1). The cell does not maintain global state about other cells, system-wide patterns, or behavior beyond its bounded scope; cells are not surveillance objects watching the system. The cell does not operate autonomously without rules; an execution context that operates on its own judgment without orchestration rules is not a cell but an autonomous agent operating on the substrate, a different and CKS-non-coherent pattern.

The six things the cell does not do are the six things most often added to cell implementations under reasonable-sounding banners — make the cell smarter, give it more context, let it remember, let it decide on its own. The architectural content of the cell layer depends on the cell not doing any of them.

4. What the cell layer is not

The four commitments place the cell as a specific architectural object distinct from four adjacent objects it is commonly conflated with. The implementation technology for any of these can host a CKS

cell when the four commitments are satisfied; what conflation costs is the architectural identity of the cell.

Not an autonomous agent. This is the most consequential conflation in current AI infrastructure discourse. An autonomous agent (in agent-framework parlance) holds goals, plans, and intermediate state across multiple steps; exercises judgment about action selection; and may operate over multiple invocations while maintaining its own state. Each property violates one or more cell commitments: goals and plans held by the agent are state external to the substrate (violating substrate mediation and statelessness); action-selection judgment exercised outside orchestration rules violates rule-governed behavior; persistent agent state across invocations violates statelessness and undermines the substrate-as-source-of-truth property statelessness supports.

The architectural difference is sharper than implementation choice. An autonomous agent wrapped with code that reads and writes a CKS substrate is not, by virtue of that wrapping, a CKS cell — it is an autonomous agent with a substrate adjacent to it. To convert the agent into a cell, four changes must occur: behavior must come under orchestration rules, scope must become bounded by rule, state must move from agent memory to substrate, and persistence across invocations must collapse into the substrate. After all four, what remains is an execution context that uses an LLM as its mediator within a cell. Many implementations make the first one or two changes and stop, producing systems that look CKS-shaped but fail cell-side commitments under load; the four-commitment specification is what makes the partial conversion identifiable as partial.

Not a microservice. A microservice is a unit of deployable code that exposes an API. A microservice can implement a CKS cell when it satisfies the four commitments, but is not a cell by virtue of being a deployable unit. A microservice that exposes an API and internally maintains its own database, decision logic, and state machine is not a cell, even if it reads from a CKS substrate as one of its data sources. Cells can run as microservices, monoliths, scripts, spreadsheet functions, or notebook cells; what makes any of them a cell is the four commitments, not the deployment shape.

Not a workflow step. A workflow step is a unit of execution in a workflow engine. A workflow step can trigger a CKS cell; the step is not the cell. The workflow engine sits at the orchestration-layer adjacency named in a separate derivation in this series, a different architectural layer from the coordination-knowledge layer where cells live. The step is the engine's representation of what to run; the cell is what runs and what it commits to architecturally.

Not a prompt template. A prompt template is a reusable LLM invocation pattern parameterized per use. A template can be part of an orchestration rule that governs a cell, but the template is not the cell. The cell is the execution context that uses the template (and the rule containing it) to produce substrate writes; the template is a piece of rule content. Conflating templates with cells loses the architectural content of the four commitments — particularly substrate mediation and bounded scope, which templates do not commit to.

5. Failure modes that violate the cell layer's identity

Each of the following names a way an implementation can call itself a cell while failing one or more of the four commitments. Each is a recognizable shape that emerges in real implementations under reasonable-sounding pressures (caching for performance, expanding scope for richer context, granting agent judgment for better decisions, allowing direct channels for lower latency).

Stateful cell — maintains substrate-relevant state between executions; statelessness is violated; substrate state lives in two places.

Unbounded-scope cell — reads from the entire substrate or claims authority over arbitrary content; bounded scope is violated; cost scales with substrate size.

Rule-bypassing cell — exercises judgment outside orchestration rules; rule-governed behavior is violated; human governance over execution is lost.

Direct-communication cell — passes state directly to another cell without going through the substrate; substrate mediation is violated at the inter-cell level; path retraceability breaks.

Authority-claiming cell — modifies content outside its rule's authorization or overrides human decisions; the human-governed commitment is violated at the cell layer.

Self-modifying cell — modifies its own orchestration rules during or between executions; the rule-authoring boundary is crossed without human authority.

Surveillance cell — maintains global state about other cells or system-wide patterns beyond its scope; bounded scope is violated; substrate-mediated coordination is replaced with cell-level surveillance.

6. Operational test

An execution context qualifies as a CKS cell if and only if all of the following are true at all times during its existence:

1. The cell's behavior is determined by human-authored orchestration rules; all cell actions are traceable to a rule.
2. The cell operates over a defined, bounded slice of substrate content for a defined task; the cell does not claim authority over arbitrary substrate content.
3. The cell reads from the substrate as its primary source of coordination state and writes its outputs back to the substrate; substrate-relevant state does not flow around the substrate.
4. The cell does not hold substrate-relevant state between its own executions; state that persists across executions lives in the substrate.
5. The cell does not exercise governance authority, modify orchestration rules, or communicate directly with other cells outside the substrate.

A representation that fails any of (1)–(5) is not a CKS cell in the architectural sense, regardless of how it is named in the system that contains it, and regardless of which technology it is implemented in. The four commitments determine cell identity; the implementation technology does not.

7. Why naming the cell layer's standalone identity matters

Implementations that conflate the cell with autonomous agents produce systems where cell behavior escapes orchestration rules. Conflating cells with microservices preserves a deployable unit but not the four architectural commitments. Conflating cells with workflow steps misframes the cell's commitments as workflow-engine features. Conflating cells with prompt templates treats execution context as rule content, silently dropping substrate mediation and bounded scope.

Naming the cell layer as a standalone architectural identity — with the four commitments specified in §2, the six restrictions in §3, and the four adjacent-object distinctions in §4 — gives downstream implementers a precise specification of what the cell side of the substrate-cell boundary commits to. Together with the substrate-side standalone treatment in the companion note, the layer-pair decomposition of the substrate-cell boundary is locked down: substrate side commits to four properties, cell side commits to four properties, and the boundary between them — the subject of the next note in this series — specifies what crosses and what does not.

Subsequent work that adopts the CKS pattern, extends it, composes it with adjacent patterns, or argues against it should use "cell" in the sense formalized here. Subsequent work that uses the term differently is using a different concept, and the difference should be named.

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems*. April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *Execution Under Rules: The Architectural Identity of the Cell Layer in the Coordination Knowledge Substrate Pattern*. 2 May 2026. ORCID: 0009-0004-8065-3235.